CME Seat and Commodities Price Forecasting

ADSP 31006

Robin Lee

2025-02-20

Forecasting CME Seat Prices

Initialize CME Data

There are three classes of seats CME, IMM, and IOM which confer the rights to trade different sets of commodities traded. CME seat owners can trade everything, IMM seat owners are allowed to trade everything except agricultural products, and IOM seat owners are allowed to trade only in index products and all options.

The seat price files are marked as `***S`. The files contain the price for which CME seats sold and the date of the sale. As expected, the seat price time series is irregularly spaced in time. The three files contain seat prices from the Chicago Mercantile Exchange.

Table 1: Sample of `cmes` Data

DateOfSale	Year	Month	price	division
1/10/2001	2001	1	188000	CME
2/21/2001	2001	2	250000	CME
3/26/2001	2001	3	250000	CME
5/17/2001	2001	5	325000	CME

Table 2: Sample of `imms` Data

DateOfSale	Year	Month	price	division
1/5/2001	2001	1	185000	IMM
1/12/2001	2001	1	180000	IMM
1/17/2001	2001	1	182500	IMM
1/24/2001	2001	1	185000	IMM

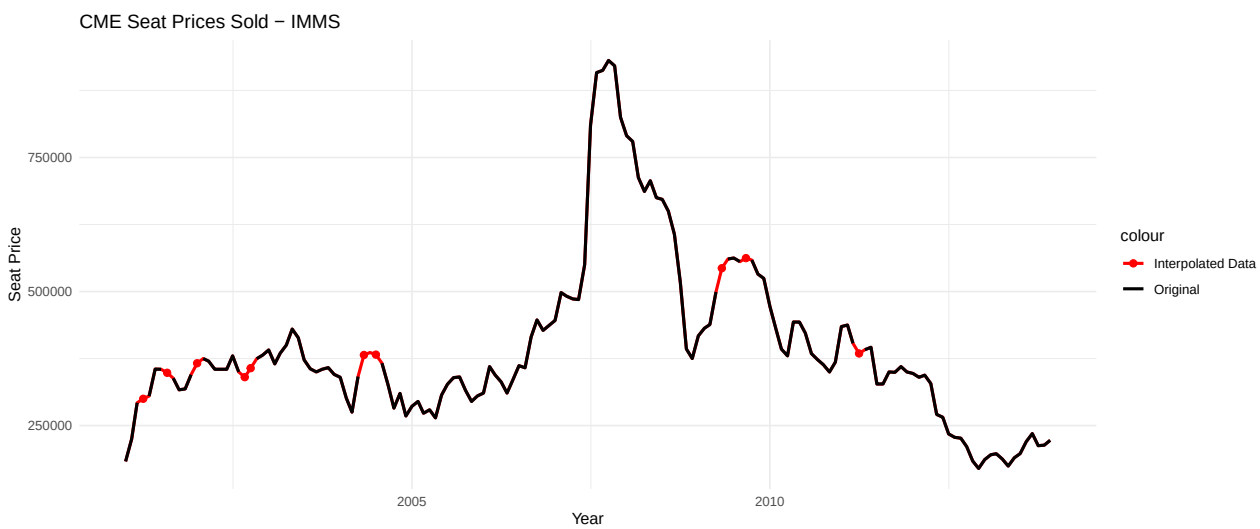
Table 3: Sample of `ioms` Data

DateOfSale	Year	Month	price	division
1/12/2001	2001	1	130000	IOM
2/13/2001	2001	2	160000	IOM
2/22/2001	2001	2	180000	IOM
3/27/2001	2001	3	235000	IOM

Data Interpolation

To create a time series that contains a seat price for each month starting from January 2001 to December 2013, I use Spline Interpolation because it is specifically geared towards imputing data with trend.

- This method works by extracting the month and year from the `DateOfSale` variable, and groups the seat price by each month. Because I seek to create a time series that contains a seat price for each month starting from January 2001 to December 2013, I average each month's price if there exists more than one. Then, I create a filler data frame with all the months from 2001 to 2013, and merge the seat data frame with the month filler.
- By doing so, I have the full month and year range in the seat data, with NA values for months without any prices. Finally, I use spline interpolation to fill any NA values with an estimated mean price. Spline interpolation can create continuous curves from discrete data points, which makes it a bit more realistic than a simple linear interpolation.



The file `contracts_volume` contains the monthly trading volume for different CME products. Each row has the first date of the following month (the trade volume is for the month), the commodity indicator for CME, the short description of the commodity, whether it is Future (F) or Option (O), the electronic trading volume and the total trading volume. The floor trading volume is the difference between the total volume and the electronic volume.

Table 4: Sample of `contracts_volume` Data

Date	Commodity.Indicator	Product.Short.Desc	Future.Option	Electronic.Volume	Total.Volume
2000-01-01	ED	EURO DLR FUT	F	232796	8379642
2000-01-01	SP	S&P 500 FUT	F	88426	1915082
2000-01-01	ES	EMINI S&P FUT	F	1302447	1305618
2000-01-01	ED	EURO DLR CALL	O	6600	972970

The file `contracts_classification` associates each CME commodity with a seat class.

Table 5: Sample of `contracts_classification` Data

Commodity.Code	Division
48	CME
56	CME
62	CME
BZ	CME

Note that owning a CME seat gives a trader advantage for floor trading. In contrast, owning a CME seat does not have any advantage for electronic trading, since electronic trading constitutes market participants trading directly with other participants via CME's network connections and order matching engine.

There are three classes of seats CME, IMM, and IOM which confer the rights to trade different sets of commodities traded. CME seat owners can trade everything, IMM seat owners are allowed to trade all futures and options except agricultural products, and IOM seat owners are allowed to trade future and options only in index products and all other CME options.

To find the volume, for example index products, add both the futures and options volume for the same index product for a month.

Data Aggregation

Commodities are traded both on the floor and electronically. Because the volume data set only has total volume and electronic volume, I still need to calculate the floor volume. Thus, I sort out of the volume data, specifically for those commodities that are relevant for the particular badge (CME, IMM, IOM). Then, I aggregate the data for each Commodity Indicator for each month, ignoring the distinctions between futures and options.

Table 6: Sample of CME Data

Date	mean_price_filled	Elec.Vol	Tot.Vol	Flo.Vol
2001-01-01	188000.0	4769234	31746144	26976910
2001-02-01	250000.0	4790165	26659092	21868927
2001-03-01	250000.0	6627259	34293767	27666508
2001-04-01	274093.7	6597591	31784964	25187373

Table 7: Sample of IMM Data

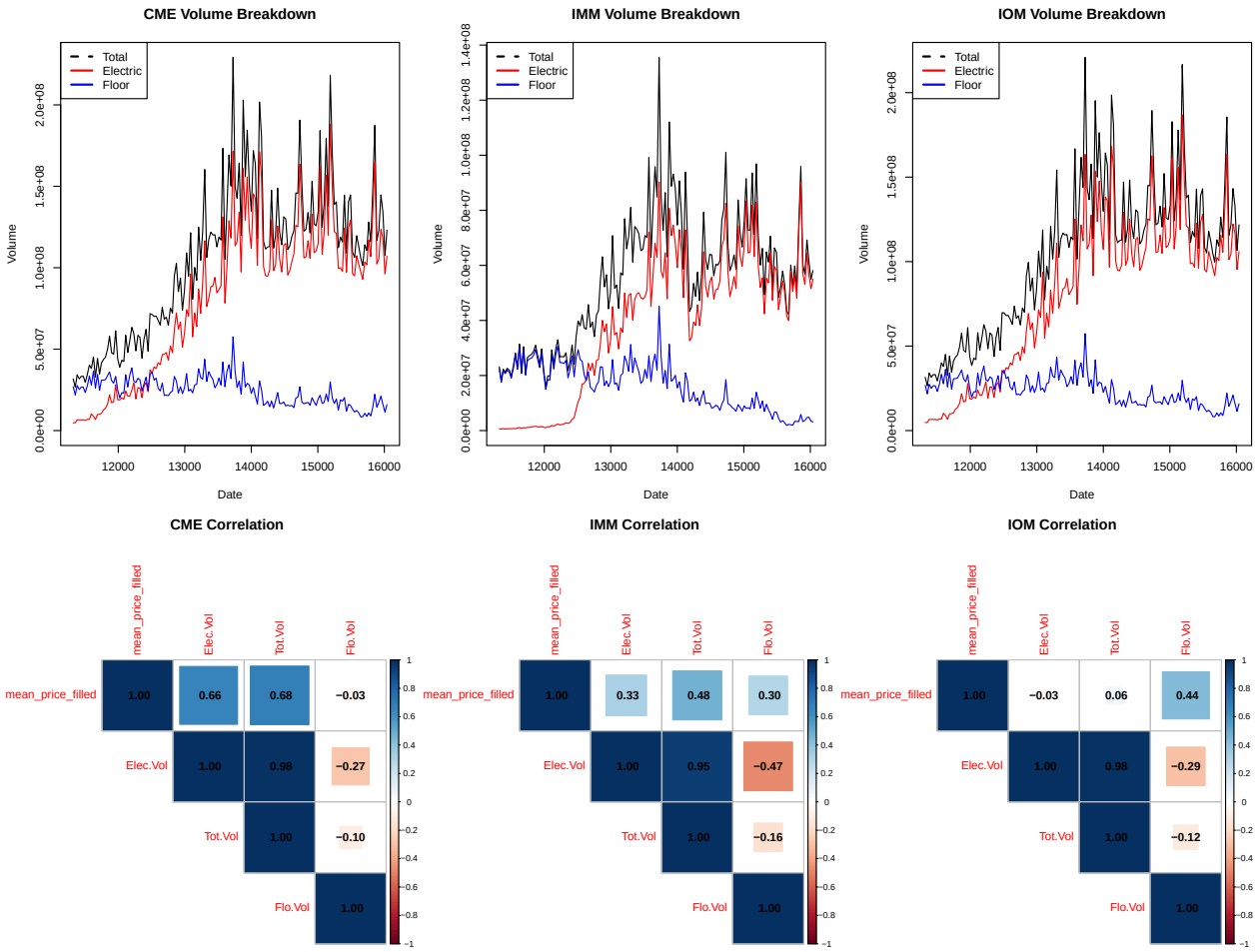
Date	mean_price_filled	Elec.Vol	Tot.Vol	Flo.Vol
2001-01-01	183125.0	619988	23140708	22520720
2001-02-01	225000.0	554640	18092448	17537808
2001-03-01	292500.0	730558	22267968	21537410
2001-04-01	299988.9	559829	21241917	20682088

Table 8: Sample of IOM Data

Date	mean_price_filled	Elec.Vol	Tot.Vol	Flo.Vol
2001-01-01	130000	4765408	31521169	26755761
2001-02-01	170000	4787364	26476344	21688980
2001-03-01	242500	6623579	34039595	27416016
2001-04-01	291500	6594361	31635884	25041523

I seek to use the trading volume information to forecast the CME monthly seat price for 2013. Following best practices, I first conduct exploratory data analysis to find initial data relationships, such as correlations. For example, the total trade volume for all CME products might be a good predictor for CME seat class, but not for the others. As such, I choose and select which commodities have the most influence on the IMM and IOM seat prices.

Exploratory Data Analysis



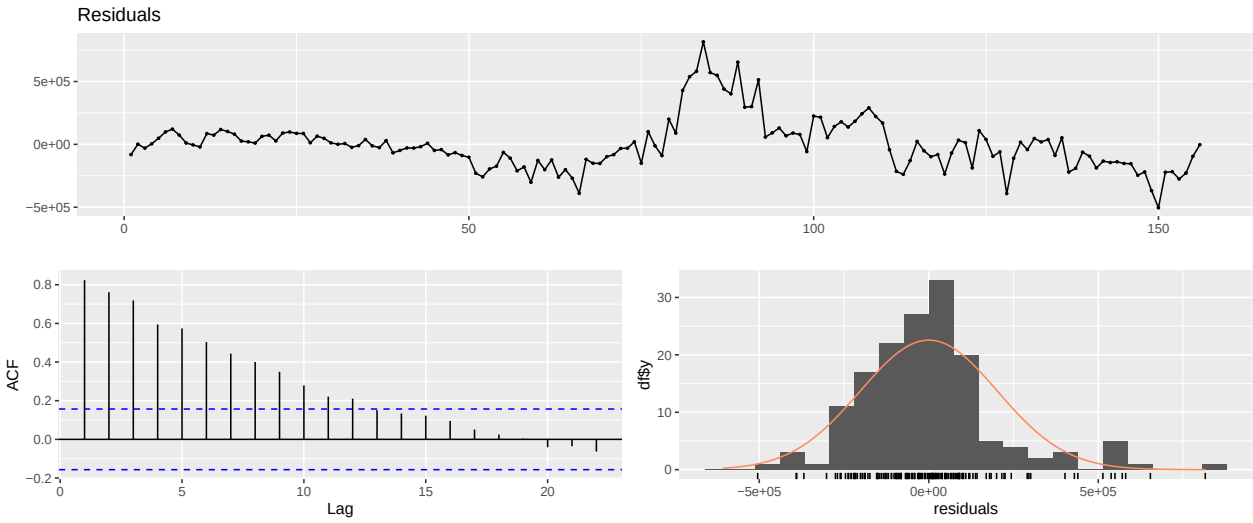
- Based off the three correlation plots above, we will use `Total Volume` for CME and IMM, as these volumes have the highest correlation. Likewise, we will use `Floor Volume` for IOM, as it is significantly higher than Electric or Total.

1. Linear regression

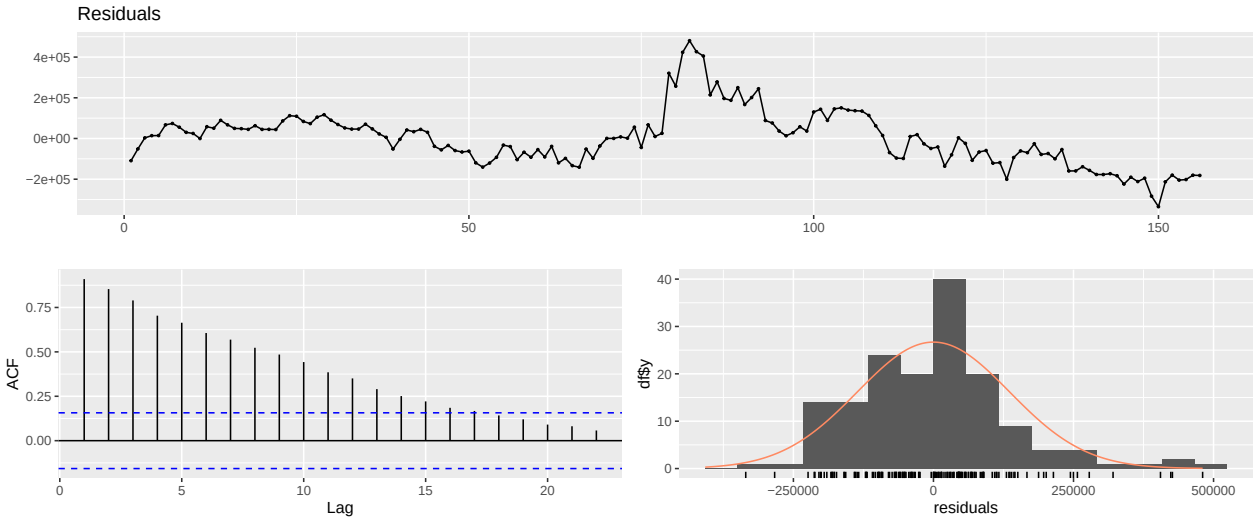
- Assuming seat price is dependent and volume(s) are independent

```
linear_regression <- function(data, label){
  if (label == "IOM"){
    model <- lm(mean_price_filled ~ Flo.Vol, data = data)
  } else {
    model <- lm(mean_price_filled ~ Tot.Vol, data = data)
  }
  return(model)
  }
```

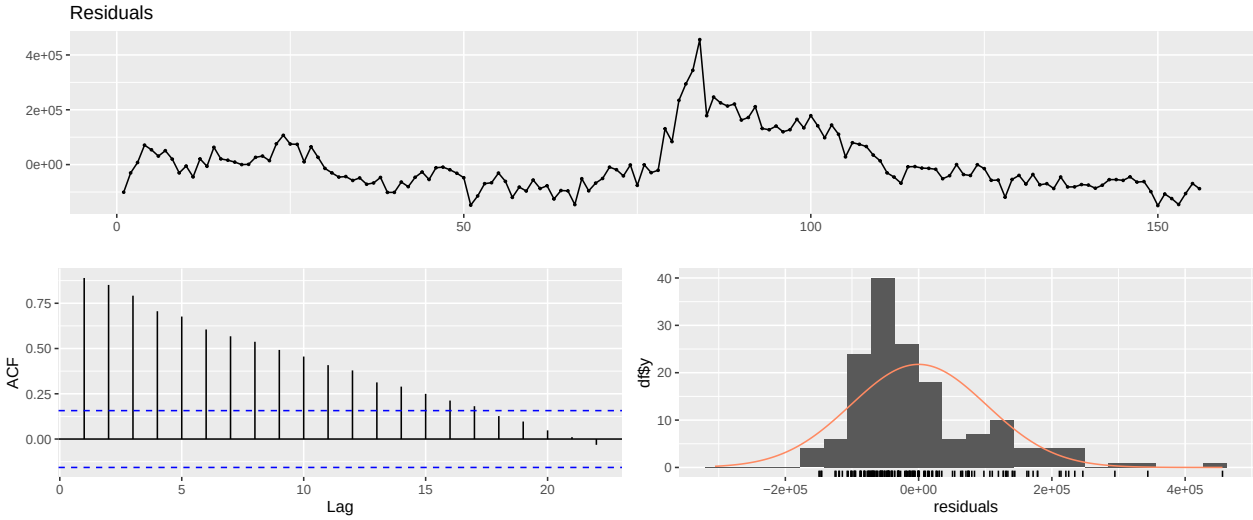
1.1. CME Division



1.2. IMM Division



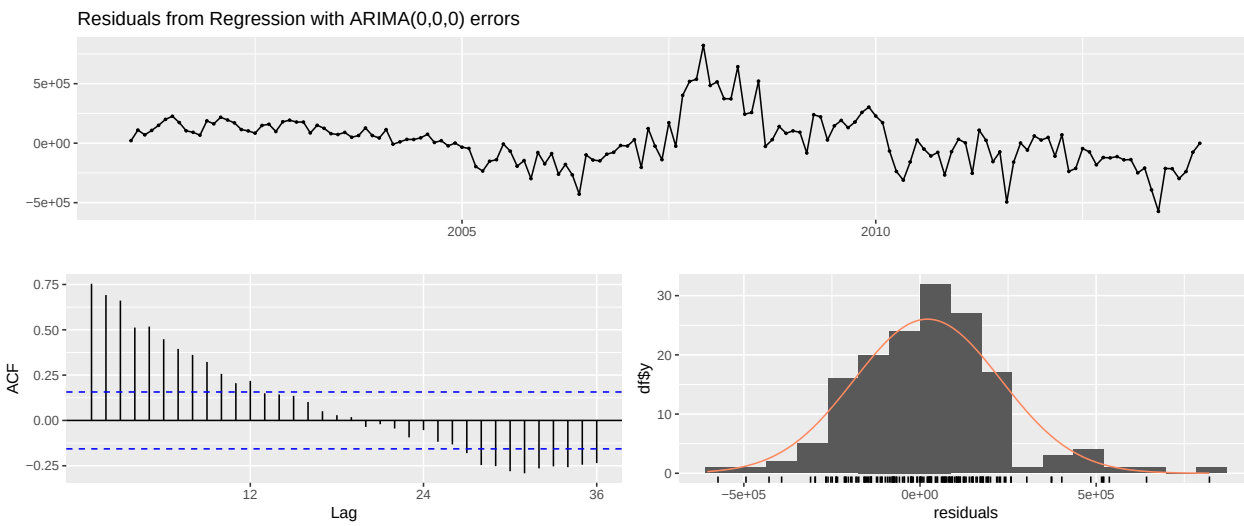
1.3. IOM Division



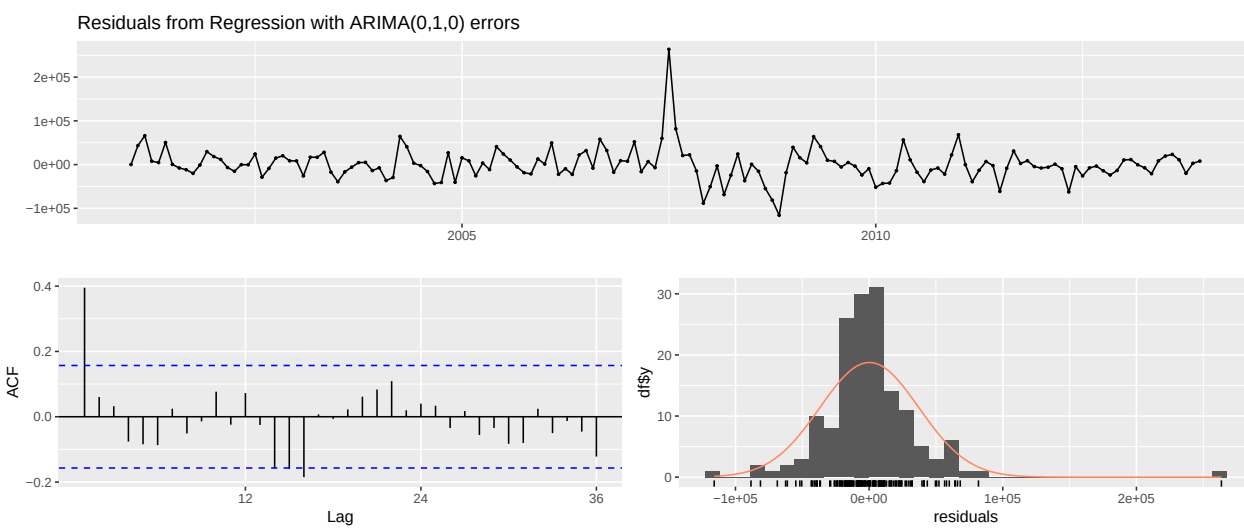
2. Linear regression with ARMA Errors

```
linear_arma <- function(data,label){
  data_ts <- ts(data$mean_price_filled, start = c(2001, 1), frequency = 12)
  if (label == "IOM"){
    xreg <- ts(data$Flo.Vol, start = c(2001, 1), frequency = 12)
  } else {
    xreg <- ts(data$Tot.Vol, start = c(2001, 1), frequency = 12)
  }
  model <- auto.arima(data_ts, xreg = xreg)
  return(model)
  }
```

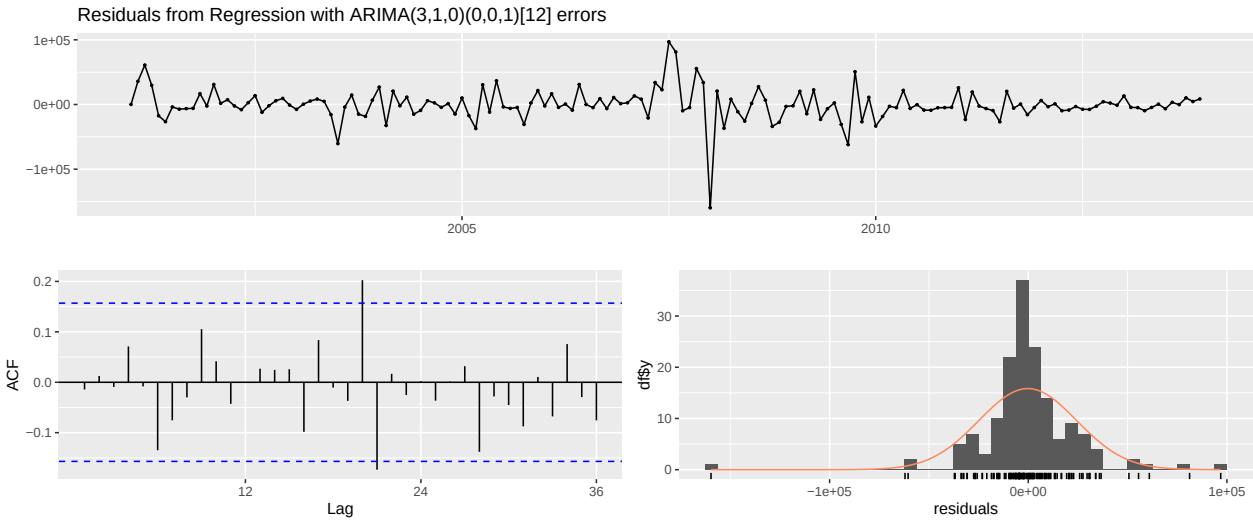
2.1. CME Division



2.2. IMM Division



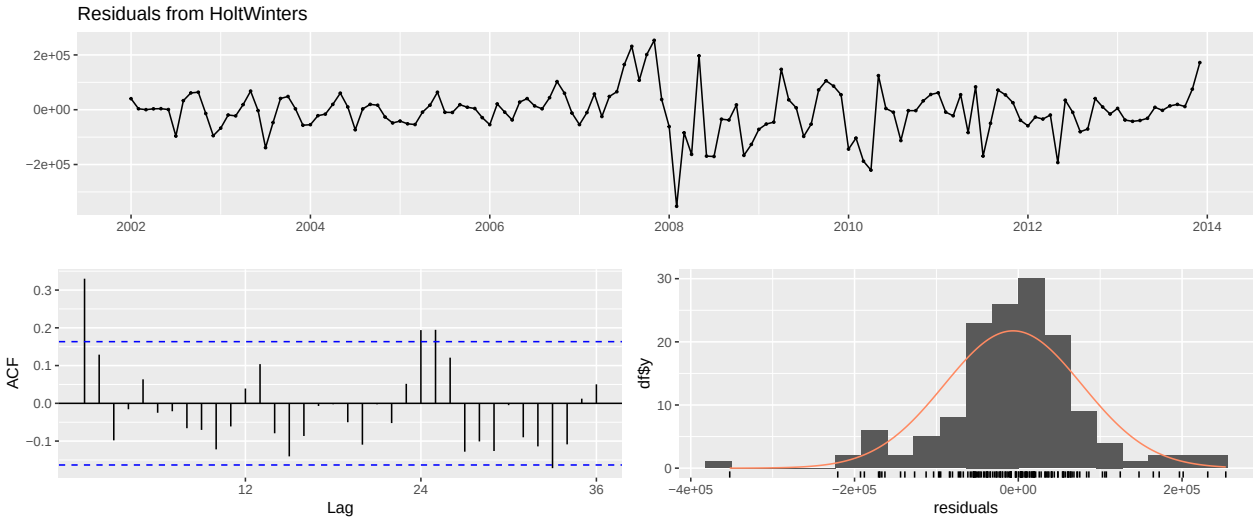
2.3. IOM Division



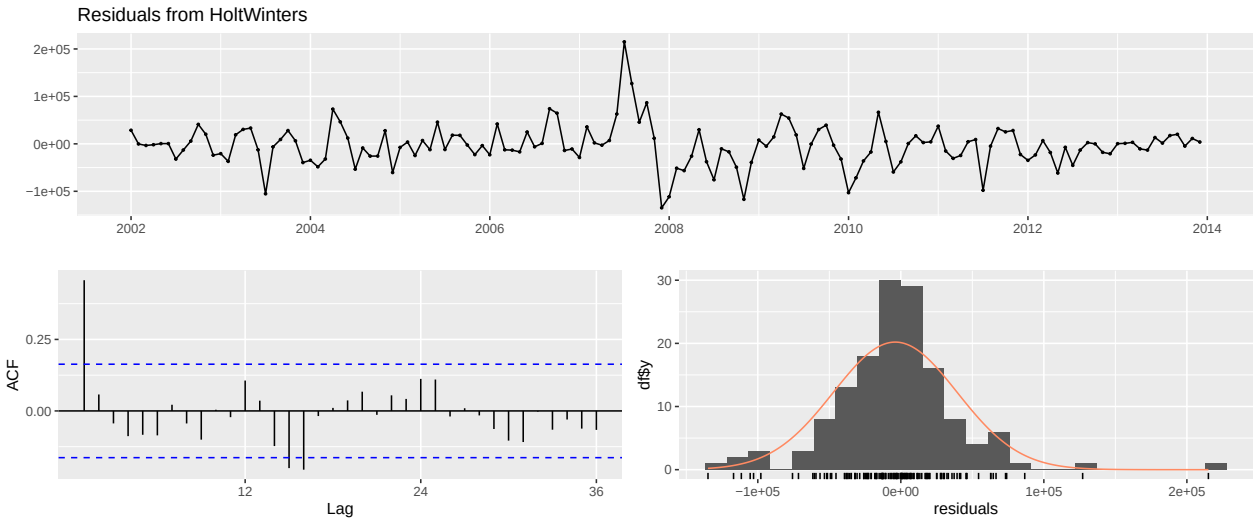
3. Holt Winters

```
holt_winters <- function(data){
  data_ts <- ts(data$mean_price_filled, start = c(2001, 1), frequency = 12)
  model <- HoltWinters(data_ts,seasonal="mult")
  return(model)
  }
```

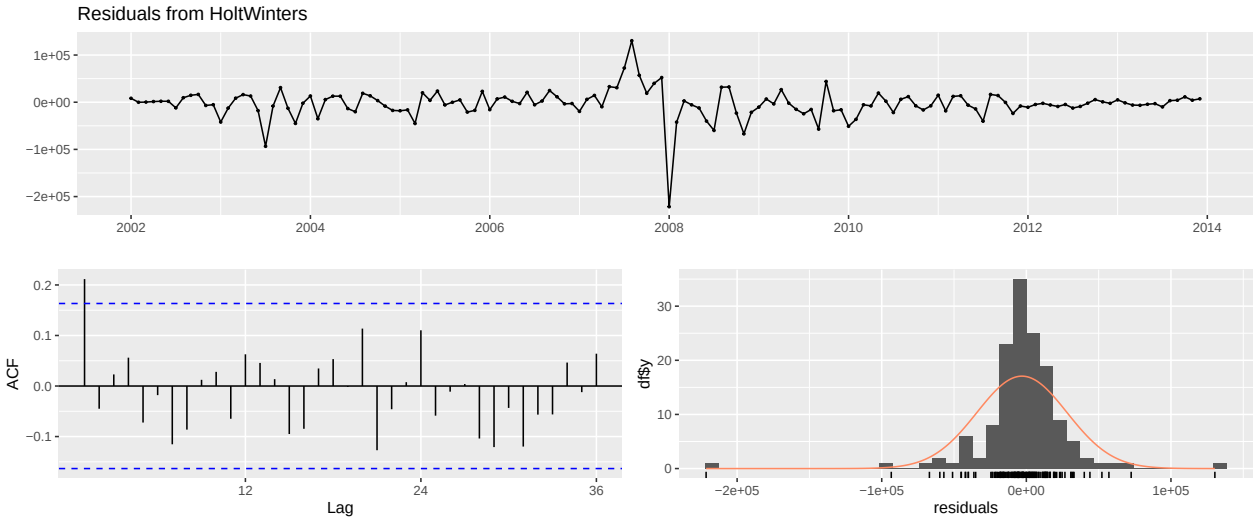
3.1. CME Division



3.2. IMM Division



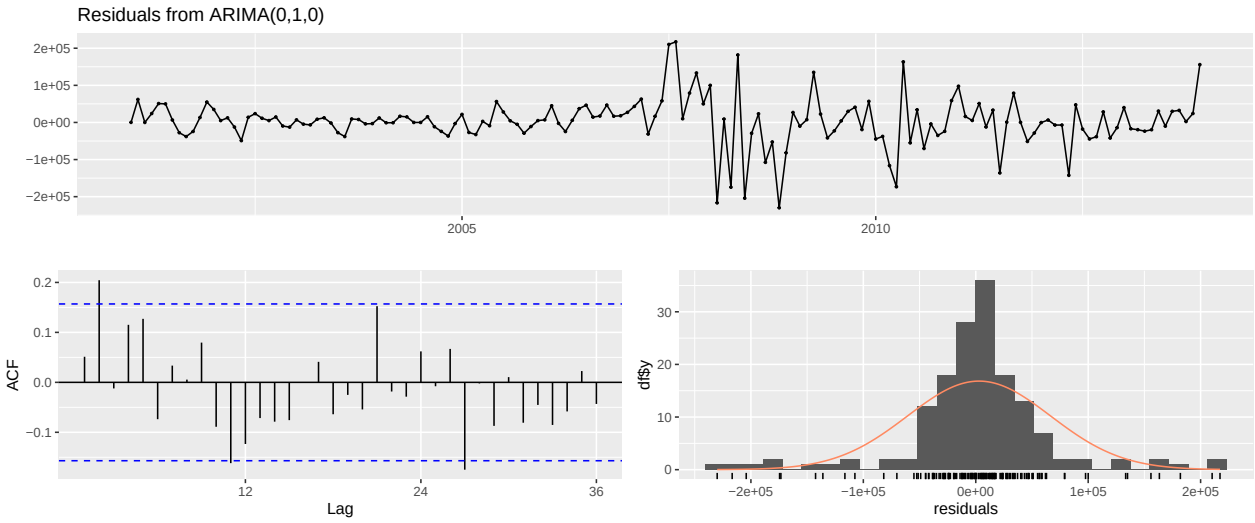
3.3. IOM Division



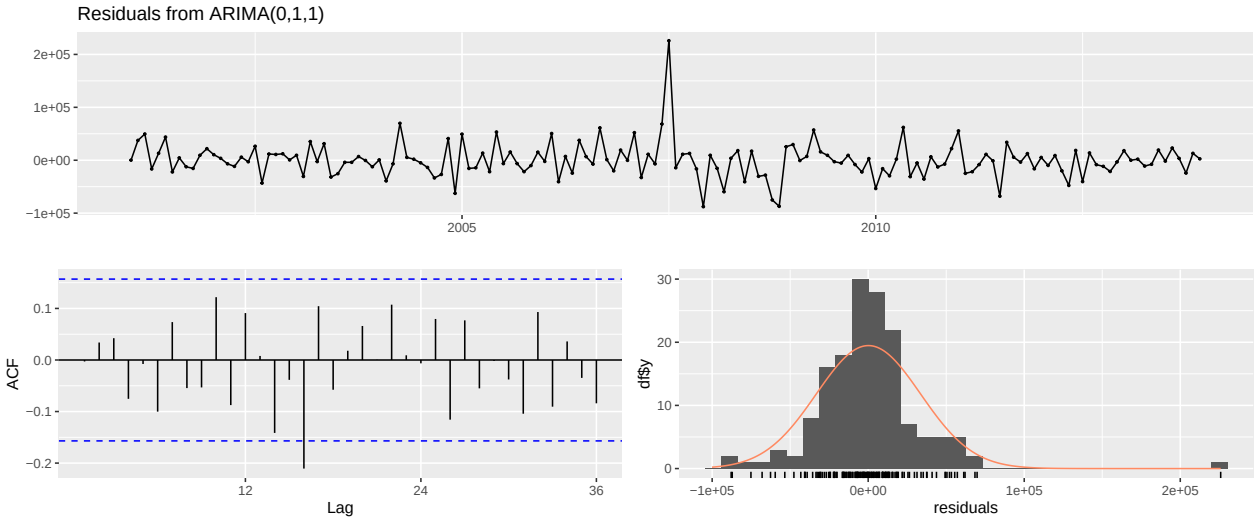
4. ARIMA

```
arima_default <- function(data){
  data_ts <- ts(data$mean_price_filled, start = c(2001, 1), frequency = 12)
  model <- auto.arima(data_ts,seasonal=FALSE)
  return(model)
  }
```

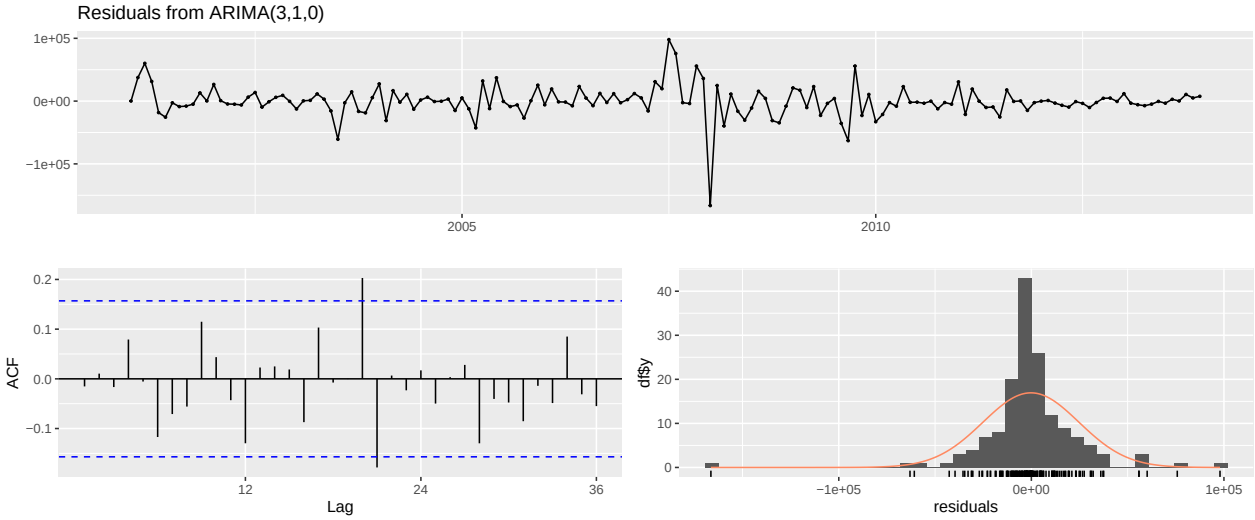
4.1. CME Division



4.2. IMM Division



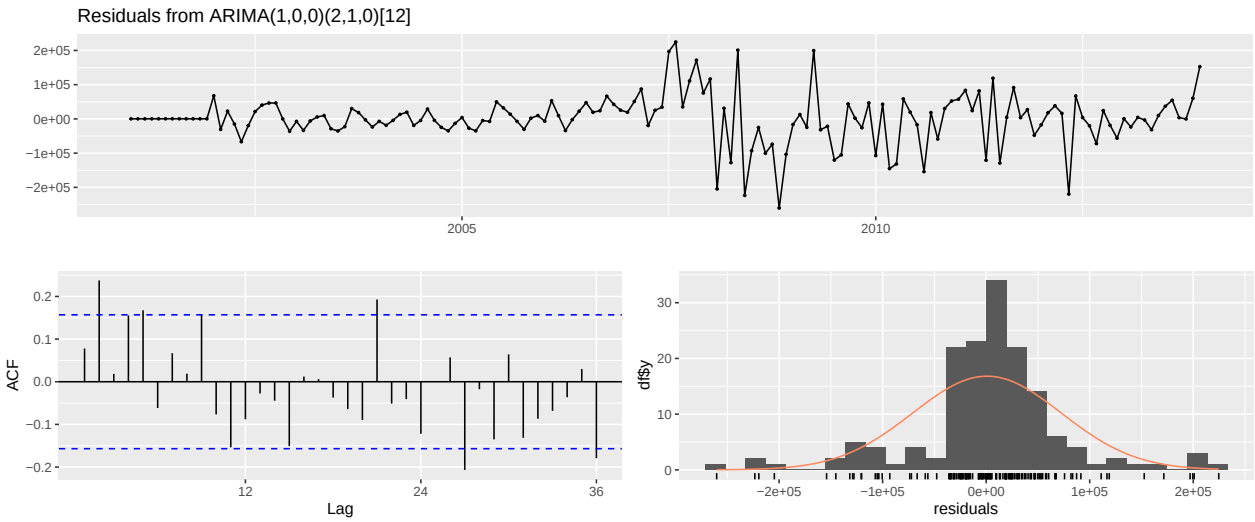
4.3. IOM Division



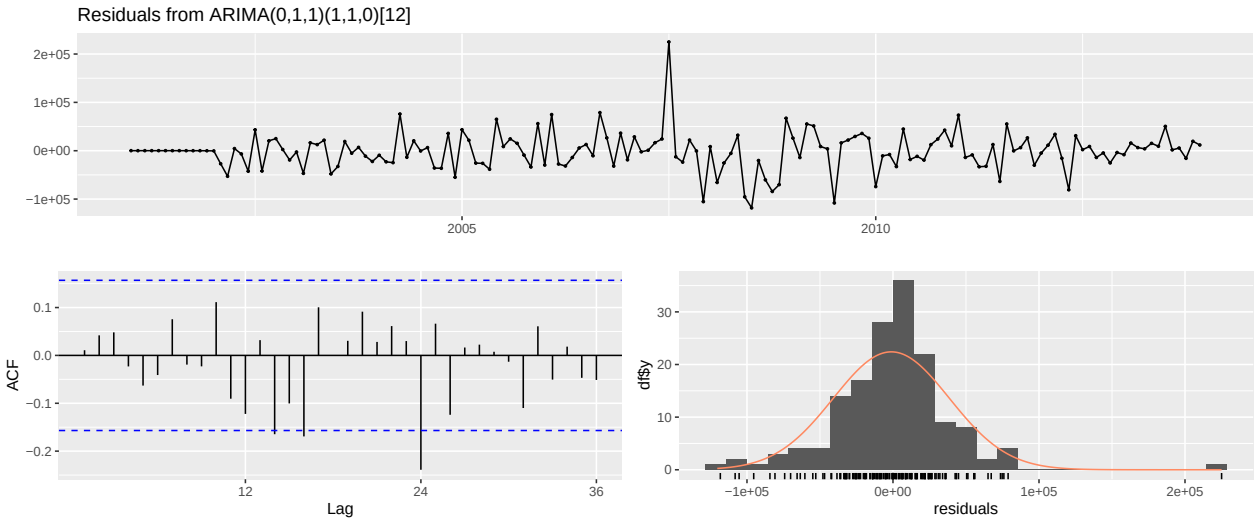
5. Seasonal ARIMA (SARIMA)

```
arima_seasonal <- function(data){
  data_ts <- ts(data$mean_price_filled, start = c(2001, 1), frequency = 12)
  model <- auto.arima(data_ts,D=1)
  return(model)
  }
```

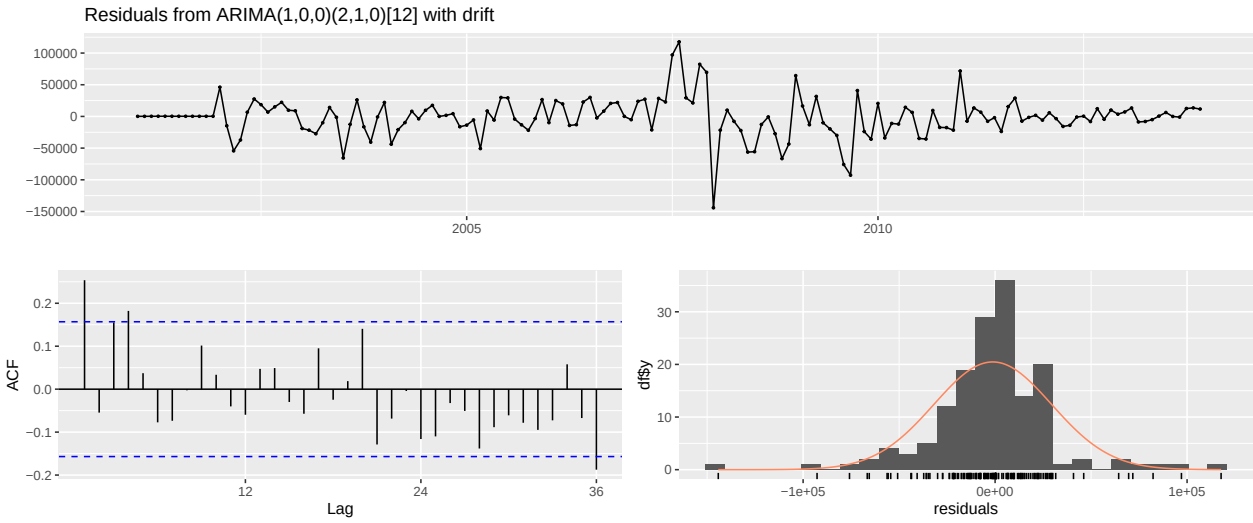
5.1. CME Division



5.2. IMM Division



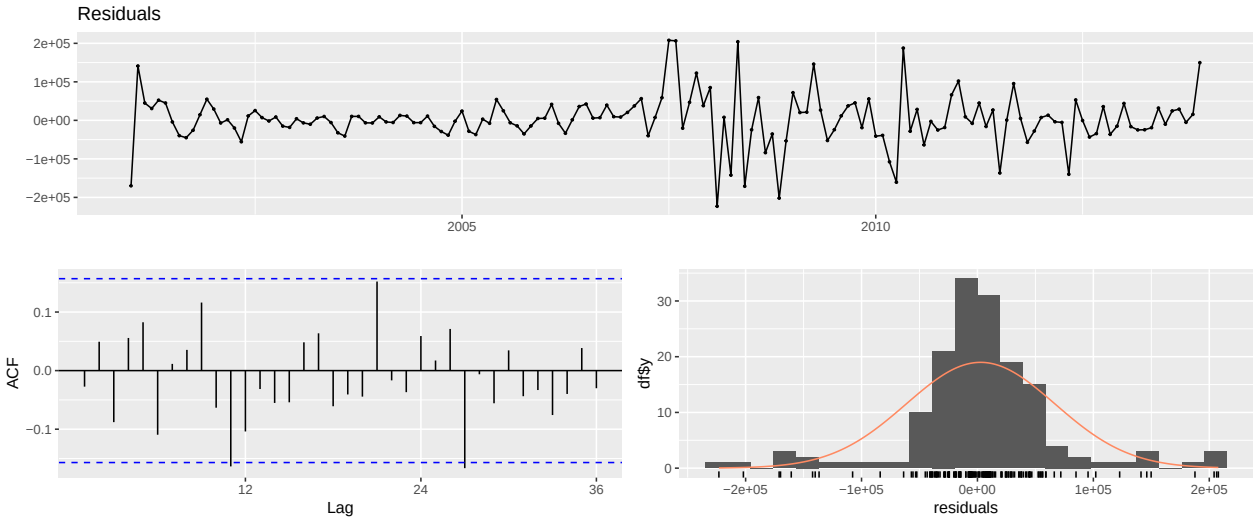
5.3. IOM Division



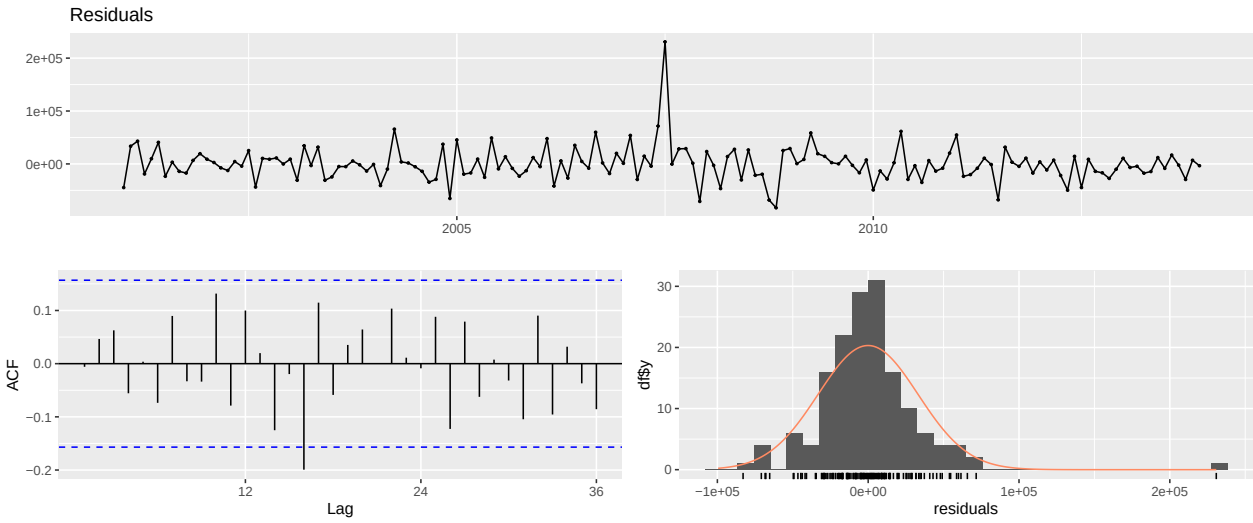
6. Fractional ARIMA (ARFIMA)

```
arima_fractional <- function(data){
  data_ts <- ts(data$mean_price_filled, start = c(2001, 1), frequency = 12)
  model <- forecast::arfima(data_ts)
  return(model)
  }
```

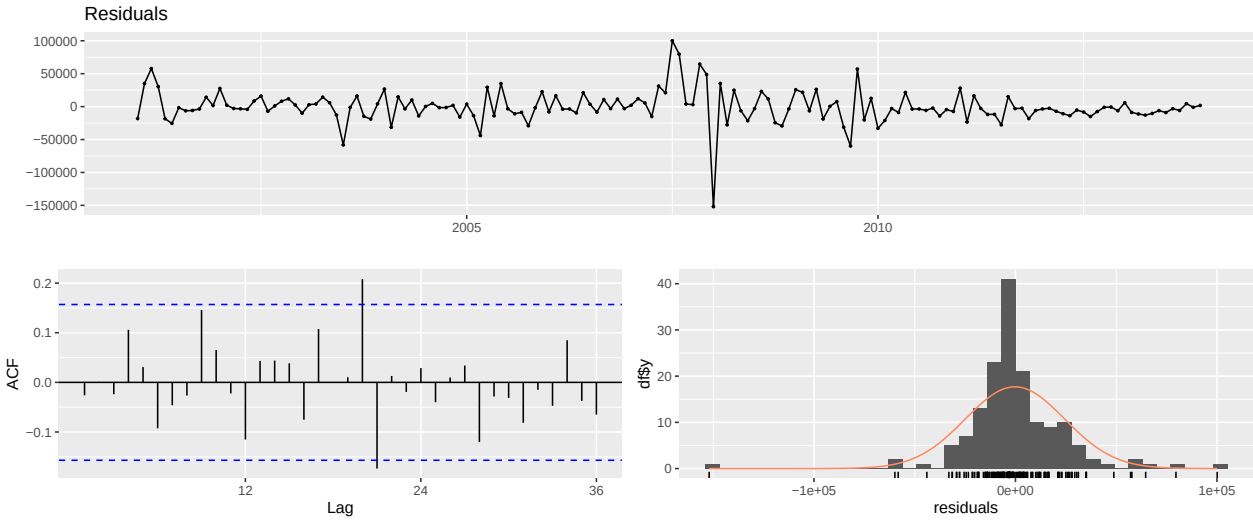
6.1. CME Division



6.2. IMM Division



6.3. IOM Division

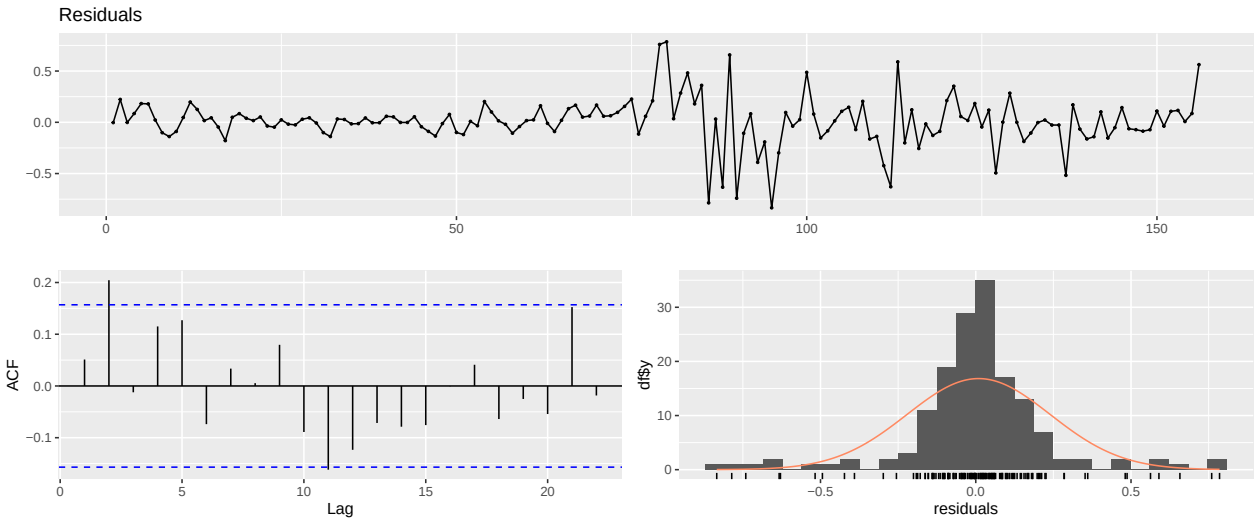


7. ARMA and GARCH combination

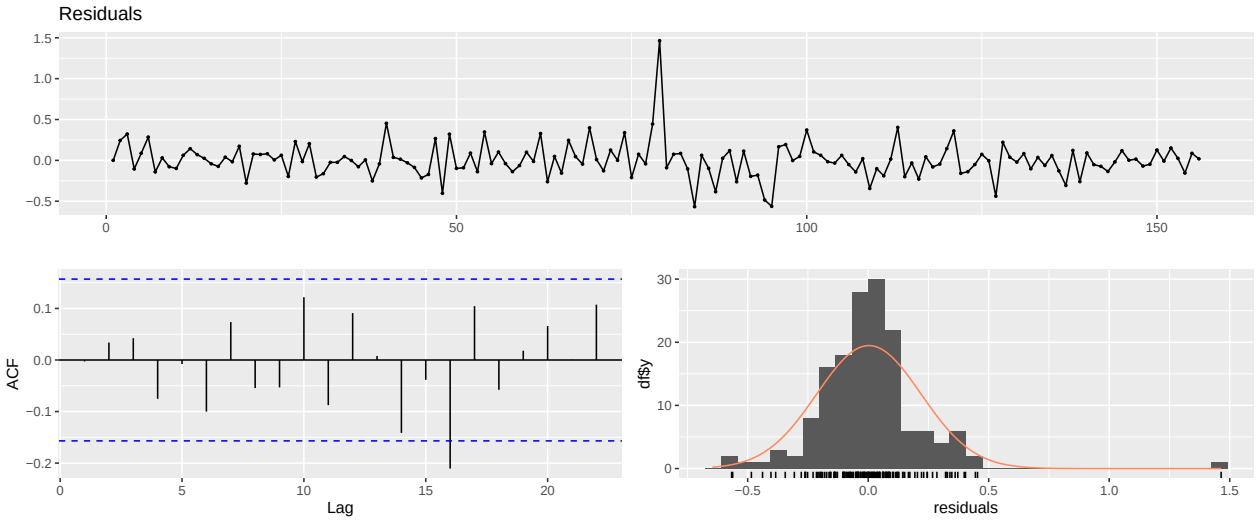
- Using the `fGarch` R library and `garchFit()`:

```
garch <- function(data){
  data$mean_price_filled <- scale(data$mean_price_filled)
  data_ts <- ts(data$mean_price_filled, start = c(2001, 1), frequency = 12)
  arma_model <- auto.arima(data_ts, seasonal = FALSE)
  garch_model <- garchFit(~ garch(1,1), data = residuals(arma_model), trace = FALSE)
  return(garch_model)
}
```

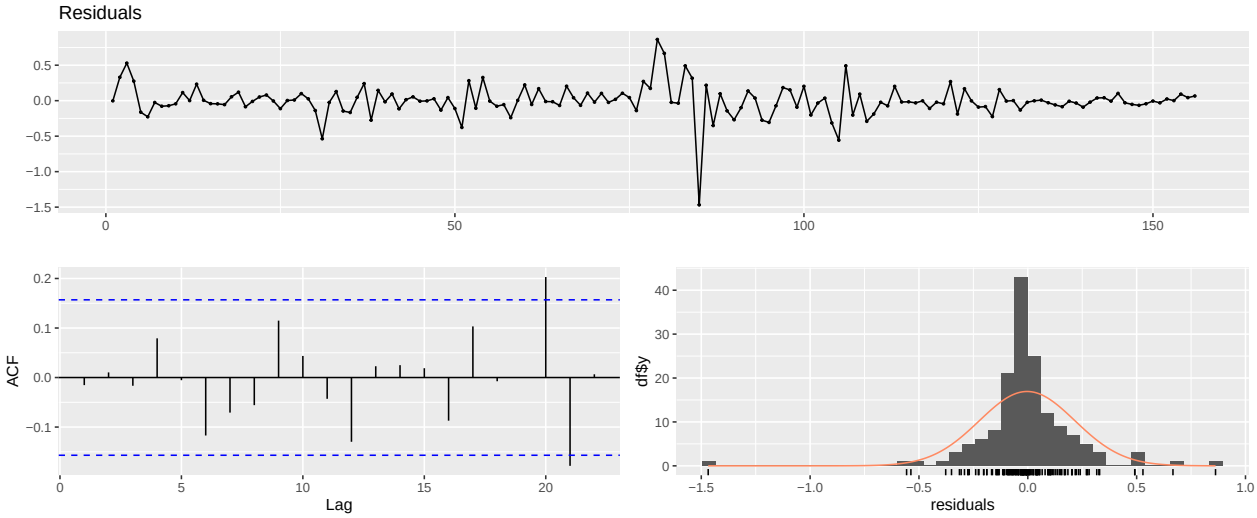
7.1. CME Division



7.2. IMM Division



7.3. IOM Division



8. Model Comparisons

8.1. CME Division

Model	RMSE
Linear Regression	280967.48
ARMA	290140.99
Holt-Winters	94116.18
ARIMA Default	73419.63
ARIMA Seasonal	89693.41
ARIMA Fractional	65526.81
GARCH	455163.55

8.2. IMM Division

Model	RMSE
Linear Regression	242626.04
ARMA	34334.20
Holt-Winters	28341.49
ARIMA Default	38311.47
ARIMA Seasonal	118742.14
ARIMA Fractional	38047.89
GARCH	203418.21

8.3. IOM Division

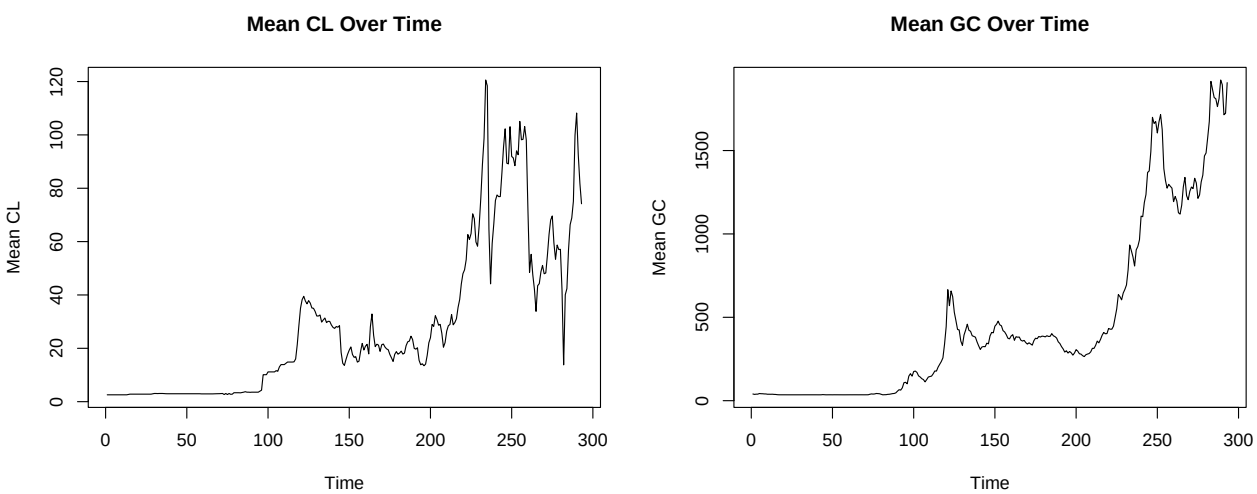
Model	RMSE
Linear Regression	173928.86
ARMA	11294.79
Holt-Winters	13269.44
ARIMA Default	10519.82
ARIMA Seasonal	11467.70
ARIMA Fractional	51737.71
GARCH	63228.61

Predicting Commodity Prices using BSTS

`sts_data_2023` is quarterly data for 2 commodities (CL & GC), 1 macroeconomic indicator (CPI) and 1 Index (SPX).

Table 12: Sample of `sts_data_2023` Data

Date	Mean_GC	Mean_CPI	Mean_CL	Mean_SPX
1950-03-31	40.15	-1.460	2.57	17.130
1950-06-30	37.75	-0.840	2.57	18.370
1950-09-30	39.15	1.895	2.57	18.065
1950-12-31	39.20	4.855	2.57	19.715



In this section, I use the BSTS package to fit models and predict CL and GC prices for 5 periods after:

- 6/1979 (fit1)
- 6/2004 (fit2)
- 12/2022 (fit3)

The models for each CL and GC use different State specifications with:

1. Dynamic regression of other 3 variables (lags)
2. Different trend component for each model

```
#data <- sts_data_2023
#date1<- "6/1979"
bsts_forecasts <- function(data, date1){
  fit_end <- ceiling_date(as.Date(paste0("01/", date1), format="%d/%m/%Y"),"month")
  data_fit <- data %>% filter(Date <= fit_end)

  # CL: Define state space specification
  data_fit_cl <- data_fit %>%
    mutate(
      GC_Lag1 = lag(Mean_GC, 1),
      CPI_Lag1 = lag(Mean_CPI, 1),
      SPX_Lag1 = lag(Mean_SPX, 1)
      ) %>%
    drop_na() # Remove NA values after lagging
```

```r
ss1_cl <- list()
ss1_cl <- AddLocalLevel(ss1_cl, y = data_fit_cl$Mean_CL)

ss2_cl <- list()
ss2_cl <- AddLocalLinearTrend(ss2_cl, y = data_fit_cl$Mean_CL)

ss3_cl <- list()
ss3_cl <- AddLocalLinearTrend(ss3_cl, y = data_fit_cl$Mean_CL)
ss3_cl <- AddSeasonal(ss3_cl, y = data_fit_cl$Mean_CL, nseasons = 4)

sink("/dev/null")
# Fit BSTS model with lags
model1_cl <- bsts(
  Mean_CL ~ GC_Lag1 + CPI_Lag1 + SPX_Lag1,
  state.specification = ss1_cl,
  data = data_fit_cl,
  niter = 1000
  )
model2_cl <- bsts(
  Mean_CL ~ GC_Lag1 + CPI_Lag1 + SPX_Lag1,
  state.specification = ss2_cl,
  data = data_fit_cl,
  niter = 1000
  )
model3_cl <- bsts(
  Mean_CL ~ GC_Lag1 + CPI_Lag1 + SPX_Lag1,
  state.specification = ss3_cl,
  data = data_fit_cl,
  niter = 1000
  )
sink()


# GC: Define state space specification
data_fit_gc <- data_fit %>%
  mutate(
    CL_Lag1 = lag(Mean_CL, 1),
    CPI_Lag1 = lag(Mean_CPI, 1),
    SPX_Lag1 = lag(Mean_SPX, 1)
    ) %>%
  drop_na() # Remove NA values after lagging

ss1_gc <- list()
ss1_gc <- AddLocalLevel(ss1_gc, y = data_fit_gc$Mean_GC)

ss2_gc <- list()
ss2_gc <- AddLocalLinearTrend(ss2_gc, y = data_fit_gc$Mean_GC)

ss3_gc <- list()
ss3_gc <- AddLocalLinearTrend(ss3_gc, y = data_fit_gc$Mean_GC)
ss3_gc <- AddSeasonal(ss3_gc, y = data_fit_gc$Mean_GC, nseasons = 4)

# Fit BSTS model with lags
```

```r
sink("/dev/null")
model1_gc <- bsts(
  Mean_GC ~ CL_Lag1 + CPI_Lag1 + SPX_Lag1,
  state.specification = ss1_gc,
  data = data_fit_gc,
  niter = 1000
  )
model2_gc <- bsts(
  Mean_GC ~ CL_Lag1 + CPI_Lag1 + SPX_Lag1,
  state.specification = ss2_gc,
  data = data_fit_gc,
  niter = 1000
  )
model3_gc <- bsts(
  Mean_GC ~ CL_Lag1 + CPI_Lag1 + SPX_Lag1,
  state.specification = ss3_gc,
  data = data_fit_gc,
  niter = 1000
  )
sink()

horizon <- 5

# CL prediction
last_values_cl <- tail(data_fit_cl, 1) # Take last available values
newdata_cl <- data.frame(
  GC_Lag1 = rep(last_values_cl$GC_Lag1, horizon),
  CPI_Lag1 = rep(last_values_cl$CPI_Lag1, horizon),
  SPX_Lag1 = rep(last_values_cl$SPX_Lag1, horizon)
  )
# Predict using BSTS
predictions1_cl <- predict(model1_cl, horizon = horizon, newdata = newdata_cl)
predictions2_cl <- predict(model2_cl, horizon = horizon, newdata = newdata_cl)
predictions3_cl <- predict(model3_cl, horizon = horizon, newdata = newdata_cl)

actual_cl <- data %>% filter(Date >= fit_end-months(2)) %>% head(5) %>%
  dplyr::select(Mean_CL)

# GC prediction
last_values_gc <- tail(data_fit_gc, 1) # Take last available values
newdata_gc <- data.frame(
  CL_Lag1 = rep(last_values_gc$CL_Lag1, horizon),
  CPI_Lag1 = rep(last_values_gc$CPI_Lag1, horizon),
  SPX_Lag1 = rep(last_values_gc$SPX_Lag1, horizon)
  )
# Predict using BSTS
predictions1_gc <- predict(model1_gc, horizon = horizon, newdata = newdata_gc)
predictions2_gc <- predict(model2_gc, horizon = horizon, newdata = newdata_gc)
predictions3_gc <- predict(model3_gc, horizon = horizon, newdata = newdata_gc)

actual_gc <- data %>% filter(Date >= fit_end-months(2)) %>% head(5) %>%
  dplyr::select(Mean_GC)
```

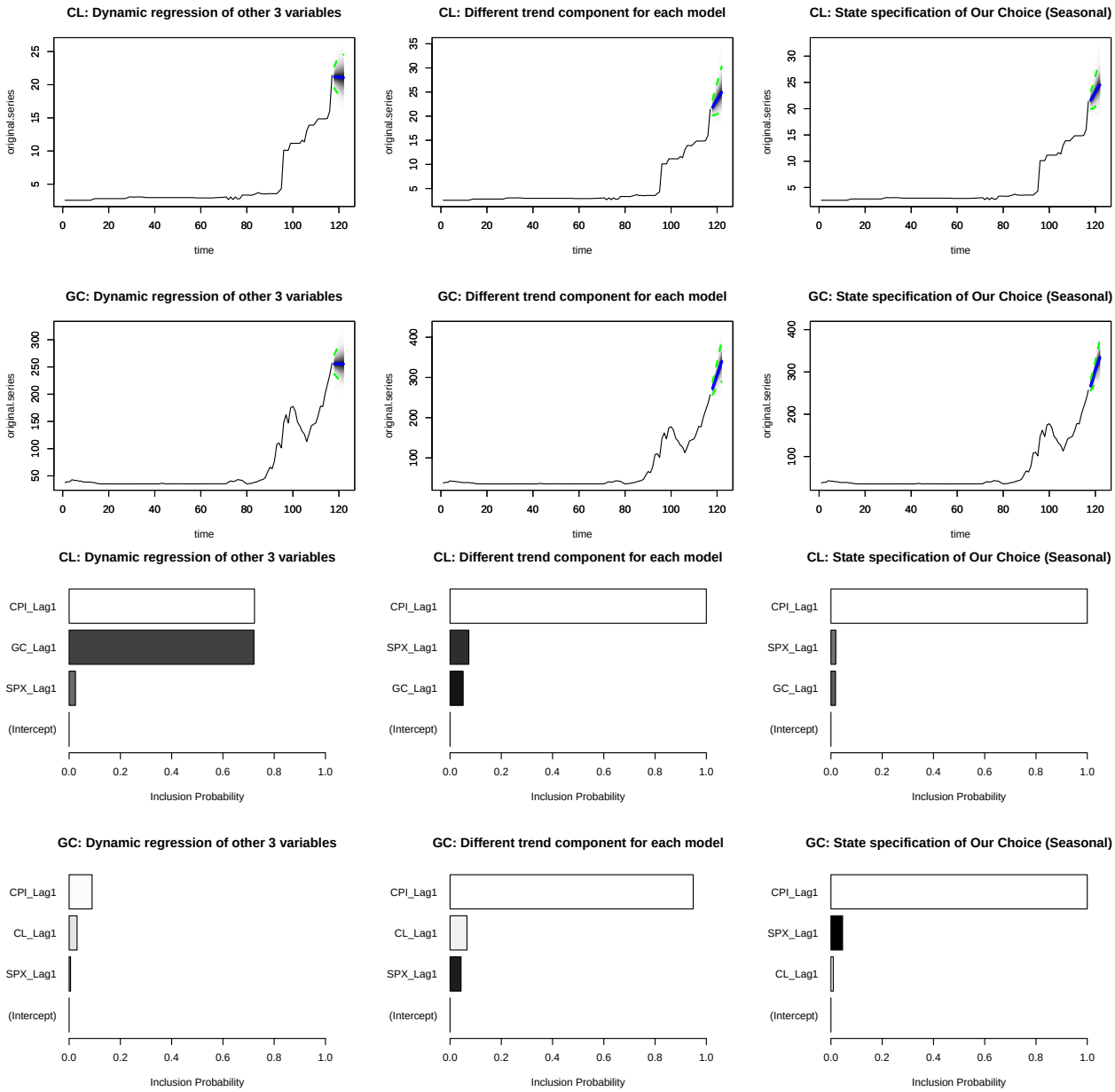
```r
  if (date1=="12/2022"){
    cat(" RMSE (CL Model 1):",Metrics::rmse(actual_cl$Mean_CL,
                                            predictions1_cl$mean[1:2]),"\n",
        "RMSE (CL Model 2):",Metrics::rmse(actual_cl$Mean_CL,
                                           predictions2_cl$mean[1:2]),"\n",
        "RMSE (CL Model 3):",Metrics::rmse(actual_cl$Mean_CL,
                                           predictions3_cl$mean[1:2]),"\n",
        "RMSE (GC Model 1):",Metrics::rmse(actual_gc$Mean_GC,
                                           predictions1_gc$mean[1:2]),"\n",
        "RMSE (GC Model 2):",Metrics::rmse(actual_gc$Mean_GC,
                                           predictions2_gc$mean[1:2]),"\n",
        "RMSE (GC Model 3):",Metrics::rmse(actual_gc$Mean_GC,
                                           predictions3_gc$mean[1:2]))
    } else {
      cat(" RMSE (CL Model 1):",Metrics::rmse(actual_cl$Mean_CL,predictions1_cl$mean),"\n",
          "RMSE (CL Model 2):",Metrics::rmse(actual_cl$Mean_CL,predictions2_cl$mean),"\n",
          "RMSE (CL Model 3):",Metrics::rmse(actual_cl$Mean_CL,predictions3_cl$mean),"\n",
          "RMSE (GC Model 1):",Metrics::rmse(actual_gc$Mean_GC,predictions1_gc$mean),"\n",
          "RMSE (GC Model 2):",Metrics::rmse(actual_gc$Mean_GC,predictions2_gc$mean),"\n",
          "RMSE (GC Model 3):",Metrics::rmse(actual_gc$Mean_GC,predictions3_gc$mean))
      }
  # Plots
  par(mfrow = c(2, 3))
  plot(predictions1_cl, main = "CL: Dynamic regression of other 3 variables")
  plot(predictions2_cl, main = "CL: Different trend component for each model")
  plot(predictions3_cl, main = "CL: State specification of Our Choice (Seasonal)")
  plot(predictions1_gc, main = "GC: Dynamic regression of other 3 variables")
  plot(predictions2_gc, main = "GC: Different trend component for each model")
  plot(predictions3_gc, main = "GC: State specification of Our Choice (Seasonal)")

  # Inclusion Probability Plots
  par(mfrow = c(2, 3))
  plot(model1_cl,"coef", main = "CL: Dynamic regression of other 3 variables")
  plot(model2_cl,"coef", main = "CL: Different trend component for each model")
  plot(model3_cl,"coef", main = "CL: State specification of Our Choice (Seasonal)")
  plot(model1_gc,"coef", main = "GC: Dynamic regression of other 3 variables")
  plot(model2_gc,"coef", main = "GC: Different trend component for each model")
  plot(model3_gc,"coef", main = "GC: State specification of Our Choice (Seasonal)")
  mtext(" ", outer = TRUE, cex = 1.5, line = -1)
}
```

Fit 1: 6/1979

```r
bsts_forecasts(sts_data_2023,"6/1979")
```

```
##  RMSE (CL Model 1): 13.27807
##  RMSE (CL Model 2): 10.67508
##  RMSE (CL Model 3): 10.99333
##  RMSE (GC Model 1): 248.57
##  RMSE (GC Model 2): 195.8078
##  RMSE (GC Model 3): 199.4523
```

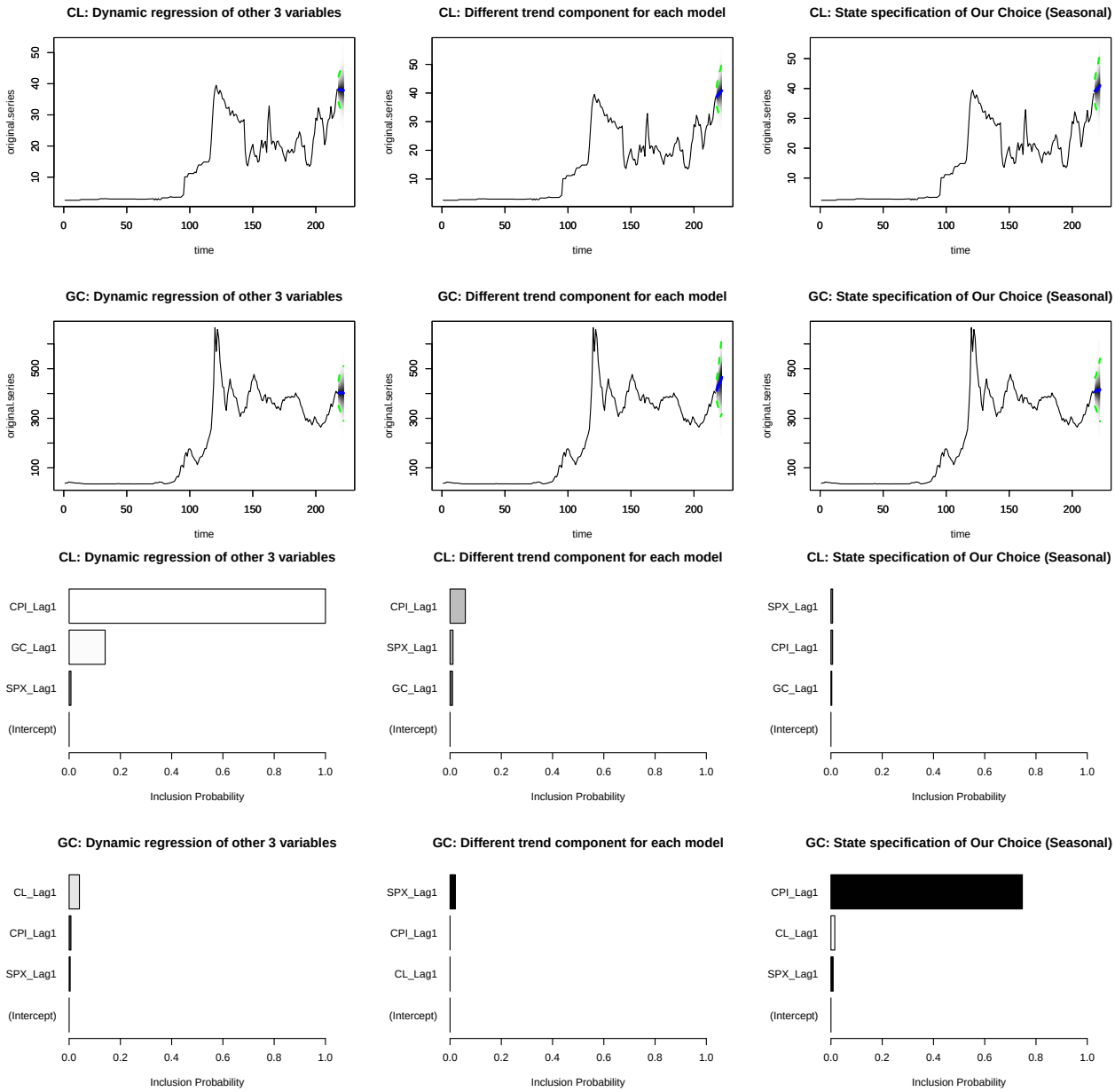


For Fit 1 (6/1979), when predicting CL, we can see that the lags of CPI are useful for Models 2 and 3, while a lag of GC is useful for Model 1. Contrastly, when predicting GC, the lags of CL are useful when using Model 1, while CPI is useful for Model 2. Finally, SPX lag is useful in predicting GC for Model 3.

Fit 2: 6/2004

```
bsts_forecasts(sts_data_2023,"6/2004")
```

```
##  RMSE (CL Model 1): 9.988413
##  RMSE (CL Model 2): 8.002416
##  RMSE (CL Model 3): 8.048744
##  RMSE (GC Model 1): 21.62274
##  RMSE (GC Model 2): 20.37502
##  RMSE (GC Model 3): 14.30431
```



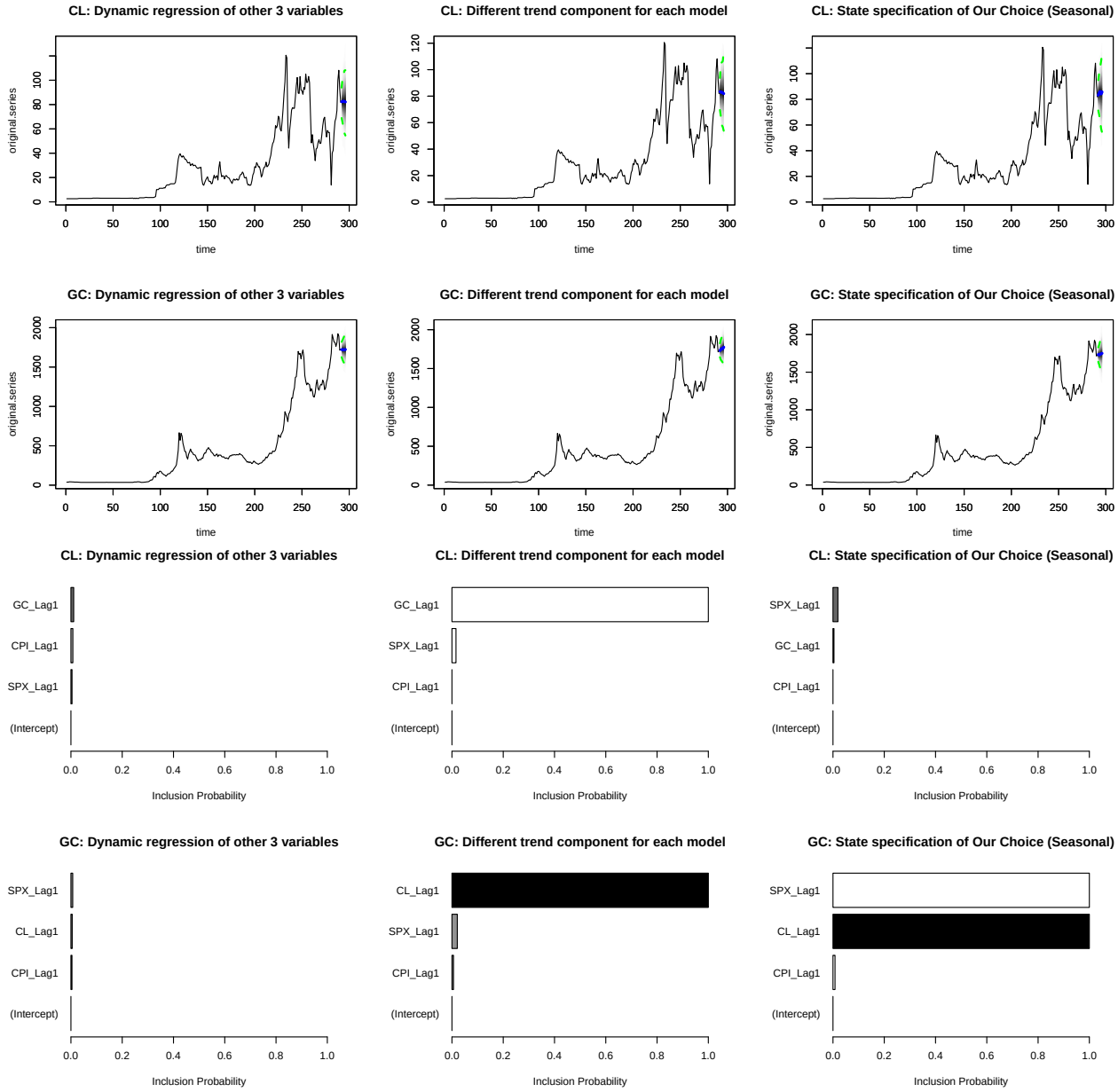
For Fit 2 (6/2004), when predicting CL, we can see that the lags of CPI and GC are useful when using Models 1 and 2, though significantly more for Model 1. It seems like lags are not very useful when using Model 3, though we see a slight increase for SPX lag. Contrastly, when predicting GC, the lags of CL are useful when using Model 1. However, lags of other variables are not very useful in predicting GC when using Model 2 and 3, though both have slight increases with SPX lag.

Fit 3: 12/2022

```
bsts_forecasts(sts_data_2023,"12/2022")
```

```
##  RMSE (CL Model 1): 6.026398
##  RMSE (CL Model 2): 6.497405
##  RMSE (CL Model 3): 7.547886
##  RMSE (GC Model 1): 131.8211
```

```
##  RMSE (GC Model 2): 113.5742
##  RMSE (GC Model 3): 124.8321
```



For Fit 3 (12/2022), when predicting CL, we see that Model 1 and Model 2 do not benefit much from lags of other variables, though both suggest CPI may be useful. However, CPI is noticeably more useful for Model 3, as well as a little from GC and SPX. Constrastly, when predicting GC, CPI lag is very useful for Model 1. Lags of other variables are not useful for Model 2, while CL lag is slightly useful in predicting GC when using Model 3.